# Report: 1 DOF Helicopter Control

**Team Members:** - Kareem Shaban 120230257 - Kamal Eldin Mohamed 120220006 - Mohamed Ashraf 120220036

**1. Introduction**

1.1. **Project Overview:** This report details the analysis, design, implementation, and testing of a control system aimed at stabilizing a Single-Input Single-Output (SISO) 1 Degree of Freedom (DOF) helicopter prototype.

1.2. **Control Objective:** The primary goal was to maintain the helicopter beam in a stable horizontal position using a Proportional-Integral-Derivative (PID) feedback control strategy implemented in LabVIEW.

1.3. **System Components Overview:** The system comprises the 1 DOF helicopter rig, a potentiometer for angle feedback, an Arduino microcontroller serving as the hardware interface, and a host PC running the LabVIEW control application utilizing the LINX toolkit.

1.4. **Report Structure:** This report covers the system analysis, the rationale behind design choices, simulation insights (or lack thereof), practical implementation details including hardware and software setup, the PID tuning process, final results, and conclusions.

---

## 2. System Analysis

### 2.1. Description of the 1 DOF Helicopter Prototype

The system under consideration is a Single-Input Single-Output (SISO) 1 Degree of Freedom (DOF) helicopter prototype. This laboratory setup typically consists of a beam mounted on a pivot, allowing it to rotate freely in a vertical plane (pitch axis). A motor with a propeller is mounted at one end of the beam, providing thrust. The rotation of the beam (its angle relative to the horizontal) is the primary variable to be controlled. The objective is to manipulate the motor thrust to stabilize the beam at a desired angle, typically horizontal.

## 2.2. Input/Output Variables

- **Input (Manipulated Variable):** The input to the system is the signal controlling the speed of the motor. In the final implementation using LabVIEW and LINX, this is represented by the pulse width signal sent to the servo/motor controller, determined by the output of the PID controller. The PID output range was set between $1100$ and 2200 microseconds in the final configuration.
- **Output (Controlled Variable):** The output is the angular position of the helicopter beam. This is measured indirectly using a potentiometer mechanically linked to the beam's pivot point. The potentiometer provides a voltage proportional to the angle, which is read by the Arduino.
- **Feedback Signal:** The voltage from the potentiometer, read via LINX Analog Read (Pin A1), serves as the process variable feedback to the PID controller. The final target setpoint was determined to be approximately 2.90 Volts, corresponding to the desired horizontal position.

## 2.3. System Dynamics (Qualitative)

The system exhibits typical dynamics of an unstable system requiring active control. Increasing the motor thrust generates an upward force, causing the beam to pitch upwards. Decreasing thrust allows gravity to pull the beam downwards. The relationship between motor input signal and beam angle is generally non-linear, and the system is susceptible to disturbances like air currents or friction in the pivot. Without control, the beam will typically fall to one end or potentially oscillate if disturbed.

## 2.4. Control Challenges

Several challenges were anticipated and encountered during the project: **Instability:** The inherent instability of the system requires continuous closed-loop control. **Non-linearity:** The relationship between motor input and aerodynamic thrust/resulting angle is not perfectly linear. **Sensor Noise:** The potentiometer, while simple, can be susceptible to noise, especially mechanical noise induced by motor vibrations, as identified during the project. **Actuator Limits:** The motor speed and corresponding pulse width signal have minimum and maximum limits (saturation). **PID Tuning:** Finding appropriate PID gains is crucial for achieving stable, accurate, and responsive control without excessive overshoot or oscillation. Initial tuning attempts resulted in instability and integral windup, requiring careful, incremental adjustments.

---

# 3. Design Rationale

## 3.1. Choice of Control Strategy: PID Control

A Proportional-Integral-Derivative (PID) control strategy was selected for this project. This is a widely used and well-understood control algorithm, particularly effective for stabilizing systems like the 1 DOF helicopter. **Proportional (P) term:** Provides an immediate corrective action proportional to the current error (difference between desired and actual angle), driving the system towards the setpoint. **Integral (I) term:** Accumulates past errors to eliminate steady-state offsets, ensuring the beam eventually settles precisely at the target angle despite constant disturbances (like gravity or slight aerodynamic imbalances). **Derivative (D) term:** Responds to the rate of change of the error, providing damping action to reduce overshoot and oscillations, leading to a smoother and more stable response. Given the need for stability, accuracy, and damping, PID control was deemed the most appropriate choice for this SISO system.

## 3.2. Choice of Feedback Sensor: Potentiometer

A potentiometer was initially chosen as the feedback sensor to measure the beam's angle. **Rationale:** Potentiometers are simple, inexpensive, and provide an analog voltage output that is easily read by microcontrollers like the Arduino. They offer a straightforward way to get position feedback. **Limitations Encountered:** During testing, it was noted that the potentiometer readings could be susceptible to noise, particularly mechanical vibrations originating from the motor. This noise could potentially degrade the control performance if not addressed.

## 3.3. Choice of Implementation Platform: LabVIEW with Arduino

LabVIEW was chosen as the primary development environment for implementing the control logic, data acquisition, and user interface. An Arduino microcontroller was used as the interface between LabVIEW and the physical hardware (potentiometer, motor). **Rationale:** LabVIEW provides a graphical programming environment well-suited for control systems, data visualization (graphs for tuning), and interfacing with hardware. The Arduino serves as a low-cost data acquisition and actuation device.

## 3.4. Choice of Communication/Interface: LINX Toolkit

Initially, standard VISA serial communication between LabVIEW and a custom Arduino sketch was considered. **Initial Consideration (VISA):** This approach involves writing custom code on the Arduino to read sensors and send data, and using LabVIEW's VISA VIs to communicate. While flexible, it requires managing serial protocols and data parsing. *

**Final Choice (LINX):** The project ultimately adopted the LINX toolkit. This toolkit provides a specific firmware for the Arduino and a library of LabVIEW VIs that allow LabVIEW to directly command Arduino functions (like Analog Read, Digital Write, Servo Write, I2C communication) without writing custom Arduino code for the main logic. **Rationale:** LINX simplifies the interaction between LabVIEW and the Arduino hardware, abstracting away the low-level serial communication details and providing dedicated VIs for hardware tasks. This allowed focusing on the control logic within LabVIEW.

### 3.5. Evolution of Design

The implementation process involved troubleshooting and design refinement: **Shift from VISA to LINX:** The initial plan based on VISA was revised in favor of LINX for easier hardware interaction within LabVIEW. **PID Tuning Challenges:** Initial PID tuning attempts with high gains led to instability and integral windup. A more systematic approach, starting with low gains, restarting the VI to clear integral windup, and incrementally adjusting P, then I, then D while observing output graphs, was adopted. * **Setpoint Determination:** The switch to LINX Analog Read (outputting voltage) required determining the voltage corresponding to the target angle (approx. 2.90V) and adjusting the PID setpoint accordingly, rather than using the raw 200-850 range from the initial Arduino tests.

---

## 4. Simulation Insights

System simulation using mathematical modeling tools (such as the LabVIEW Control Design and Simulation Module) was not performed as part of this specific project implementation.

The control system development, including the implementation of the PID controller and the tuning of its parameters, was conducted directly on the physical hardware prototype. The tuning process relied on observing the real-time response of the system through graphical indicators in LabVIEW and iteratively adjusting the gains based on the observed performance (stability, overshoot, settling time, steady-state error).

---

## 5. Practical Implementation Details

### 5.1. Hardware Setup

- **Microcontroller:** An Arduino UNO board was used as the interface hardware.

- **Computer Connection:** The Arduino was connected to the host PC running LabVIEW via a standard USB cable, appearing as a serial port .
- **Feedback Sensor:** A potentiometer was mechanically coupled to the pivot axis of the 1 DOF helicopter beam. The potentiometer's wiper output was connected to Analog Input Pin A1 on the Arduino.

**Actuator:** A servo motor (controlled via pulse width modulation) was used to drive the propeller, providing thrust. The servo control signal pin was connected to Arduino Digital Pin 9 (as indicated by the "Servo Channel" input to the LINX Servo VI, assuming standard LINX pin mapping).

## 5.2. Software Setup

- **Arduino Firmware:** The standard LINX firmware was uploaded to the Arduino board using the LabVIEW LINX Firmware Wizard. This allows LabVIEW to directly interact with the Arduino's I/O pins.
- **LabVIEW Environment:** LabVIEW (version not specified) was used for developing the control application.
- **Required Toolkits:** The LabVIEW MakerHub LINX toolkit was required and utilized for interfacing with the Arduino.

## 5.3. LabVIEW Implementation

The control logic was implemented within a LabVIEW VI, utilizing the LINX toolkit. The core structure involved: **Initialization (Outside Loop):** A `LINX Open` VI established communication with the Arduino on COM7. **Main Loop (While Loop):** Contained the real-time control logic. **Shift Registers:** Used to pass the `LINX Resource` and `Error Cluster` through loop iterations. **Feedback Reading:** A `LINX Analog Read` VI was configured to read from AI Channel 1 (Pin A1), outputting the potentiometer voltage (Process Variable). **PID Controller:** A standard LabVIEW `PID` VI calculated the control output based on the Process Variable and the Setpoint (target voltage, set to 2.90V). * Inputs: Process Variable (from Analog Read), Setpoint (Front Panel Control), PID Gains (Front Panel Cluster), Output Range (Front Panel Cluster: Low=1400, High=2200). * Output: Calculated control effort (pulse width). **Actuation:** The PID output was wired to a `LINX Servo` VI (implied, as Servo Channel 9 is specified). This VI sends the appropriate PWM signal to Digital Pin 9 to control the motor/servo. **Timing:** A `Wait` VI (implied, or using `dt` input for `PID`) controlled the loop execution rate. **Stop Condition:** A Front Panel "Stop" button controlled the While Loop termination. **Closing (After Loop):** A `LINX Close` VI terminated the connection to the Arduino gracefully. **User Interface (Front Panel):** Included controls for Setpoint, PID Gains (P, Ti, Td), Output Range, Servo Channel, Serial Port selection, and Stop button. Indicators displayed the

PID output, loop timing ( `dt out` ), and potentially graphs for error or output (added during tuning).

### 5.4. PID Tuning Process

The PID controller was tuned manually using the Front Panel controls while observing the system's real-time behavior, likely aided by output graphs. * **Initial Challenges:** Early tuning attempts faced issues with instability and integral windup due to overly aggressive initial gain settings. **Methodology:** A systematic approach was adopted: 1. Restarting the VI to clear the PID state (especially the integral term). 2. Starting with low P gain and zero I and D gains. 3. Incrementally increasing P gain to improve response speed without causing significant oscillation. 4. Incrementally decreasing Integral Time (Ti) to reduce steady-state error without causing excessive overshoot. 5. Introducing Derivative Time (Td) to dampen any remaining oscillations or overshoot. **Final Gains:** The process resulted in the final tuned gains: P (Kc) = 3.800, Ti = 0.038 min, Td = 0.002 min.

### 5.5. Final System Performance

As demonstrated in the provided video
, the final implementation with the tuned PID controller successfully stabilized the 1 DOF helicopter beam around the desired horizontal position. The system responded to disturbances and maintained stability, indicating the control objectives were met using the potentiometer feedback and the LINX-based LabVIEW implementation.

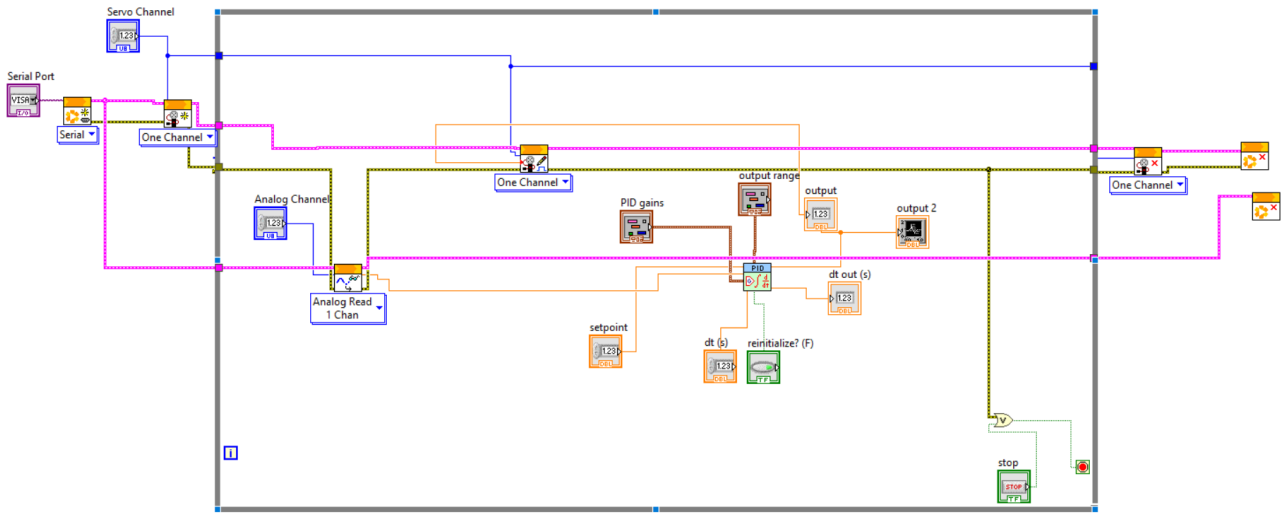
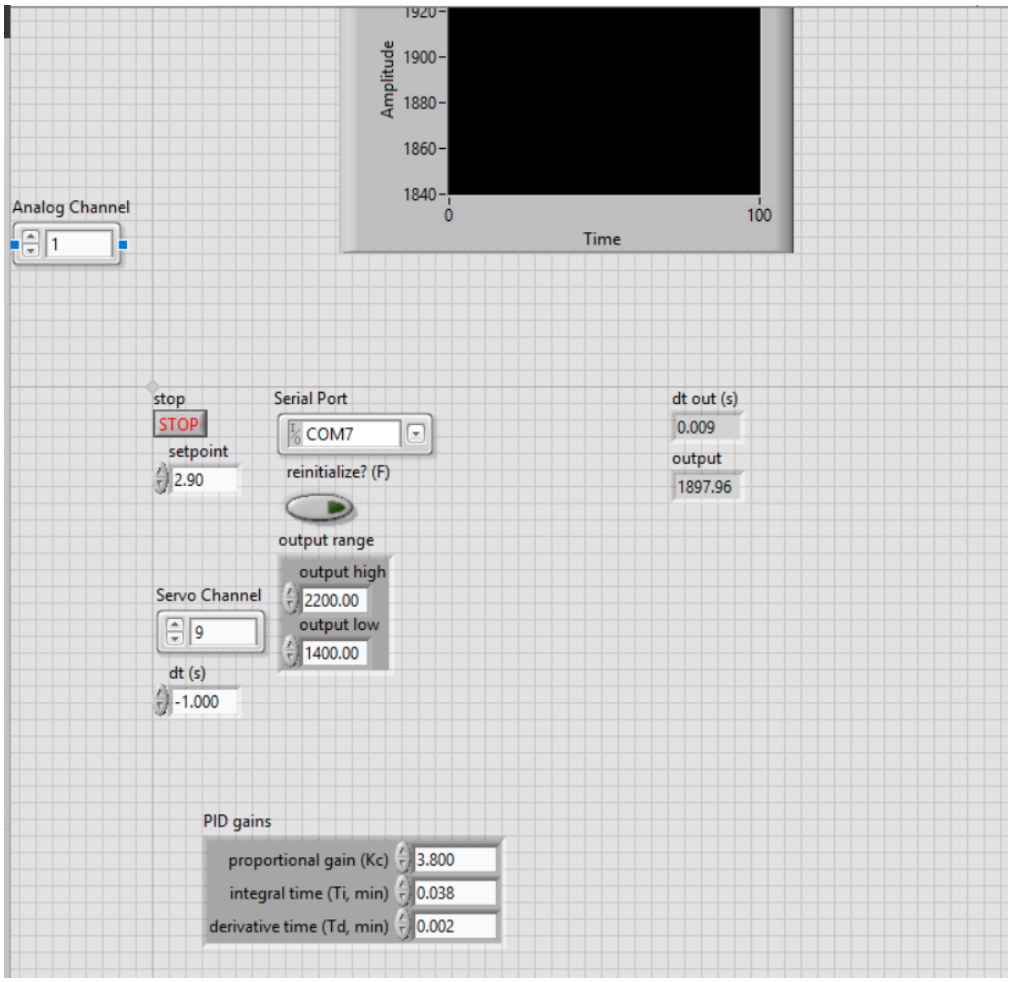
---

## 6. Conclusion

6.1. **Summary of Project Achievements:** This project successfully implemented a PID control system in LabVIEW using the LINX toolkit to stabilize a 1 DOF helicopter prototype. Feedback was obtained using a potentiometer, and the controller was tuned manually to achieve stable performance.

6.2. **Discussion of Results vs. Objectives:** The primary objective of maintaining the beam in a stable horizontal position was achieved, as demonstrated by the working prototype and video evidence. The PID controller effectively managed the system's inherent instability.

6.3. **Challenges Encountered and Solutions Implemented:** Key challenges included initial PID tuning difficulties (instability, integral windup) which were overcome by adopting a systematic, incremental tuning approach and restarting the VI to clear the

controller state. The potential for sensor noise due to vibration was noted, although the final tuning achieved acceptable stability. The switch from a potential VISA implementation to LINX simplified hardware interaction.

6.4. **Potential Future Improvements:** Further work could involve exploring alternative feedback sensors like an IMU to potentially improve noise immunity and provide direct angle measurement. System identification could be performed to create a mathematical model, enabling simulation and model-based control design techniques.